

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность
Работа №1
«Разработка защищенного REST API с интеграцией в CI/CD»

Саранча Павел Александрович
Группа: Р3409

2026

1. Сдаваемые материалы

Материал	Ссылка
Публичный git-репозиторий	https://github.com/PaulLocust/CS-lab1
Файл README.md	https://github.com/PaulLocust/CS-lab1/blob/main/README.md
Последний успешный запуск pipeline	https://github.com/PaulLocust/CS-lab1/actions/runs/35995043294
Все запуски workflow	https://github.com/PaulLocust/CS-lab1/actions/workflows/ci.yml

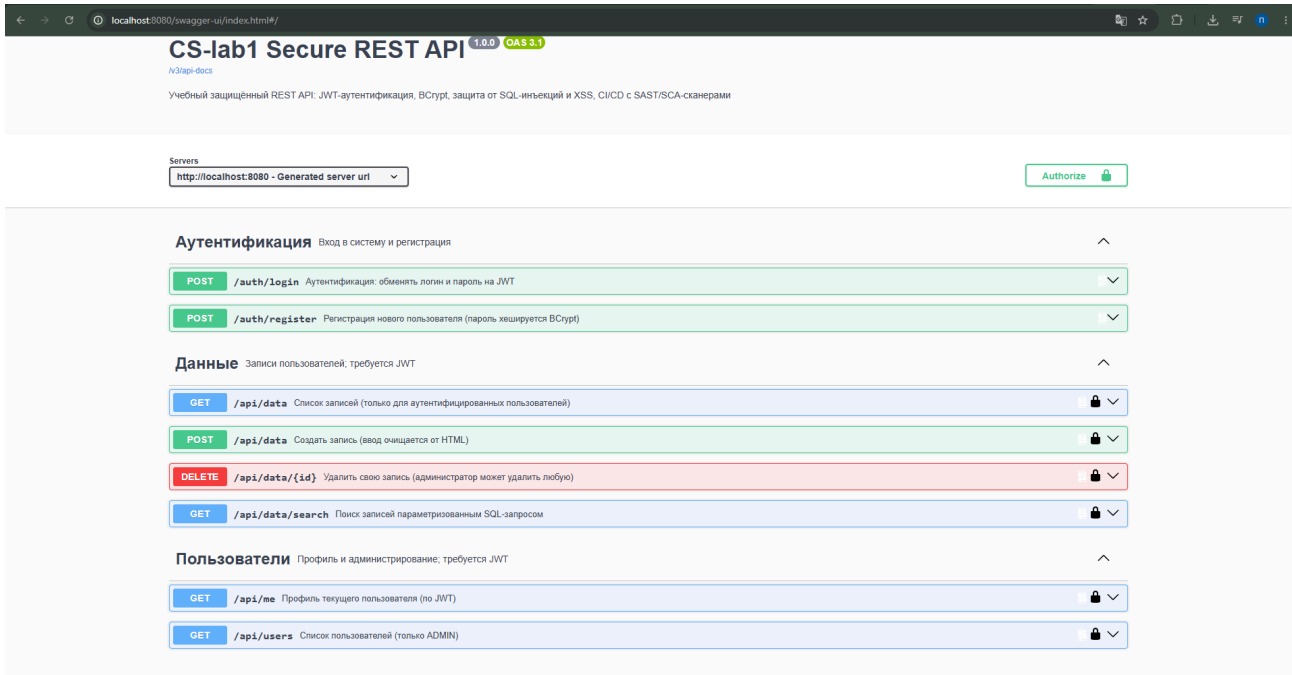
2. Описание проекта

Учебный backend-сервис — REST API для работы с записями пользователей. Проект демонстрирует защиту от основных рисков OWASP Top 10 2021 (A01 Broken Access Control, A03 Injection, A07 Identification and Authentication Failures) и автоматическую проверку кода и зависимостей в GitHub Actions.

Компонент	Выбор	Назначение
Язык и сборка	Java 21, Maven (+ Maven Wrapper)	требование задания; wrapper позволяет собрать проект без установленного Maven
Фреймворк	Spring Boot 3.5.16, Spring Security 6	безопасные значения по умолчанию
База данных	H2 (in-memory) + Spring Data JPA	ORM даёт параметризованные запросы
Аутентификация	JWT 0.12.7 (HS256), BCrypt cost 12	stateless-токены, медленное хеширование паролей
Защита от XSS	Jsoup 1.23.2, OWASP Java Encoder 1.3.1	очистка ввода и экранирование вывода
SAST	SpotBugs 4.9 + FindSecBugs 1.14, CodeQL	статический анализ кода
SCA	OWASP Dependency-Check 13, Trivy, Dependabot	поиск известных CVE в зависимостях
Документация API	springdoc-openapi (Swagger UI)	интерактивная проверка эндпоинтов

3. Описание API

Три обязательных метода по заданию — POST /auth/login, GET /api/data и придуманный самостоятельно POST /api/data. Остальные добавлены, чтобы показать разграничение прав и работу с профилем.



№	Метод и эндпоинт	Доступ	Назначение
1	POST /auth/login	публичный	аутентификация, выдача JWT
2	GET /api/data	JWT	постраничный список записей
3	POST /api/data	JWT	свой метод: создание записи с очисткой ввода от HTML
4	GET /api/data/search	JWT	поиск через JDBC PreparedStatement
5	DELETE /api/data/{id}	JWT, автор или ADMIN	удаление записи с проверкой владельца
6	GET /api/me	JWT	профиль владельца токена
7	GET /api/users	JWT + роль ADMIN	список пользователей
8	POST /auth/register	публичный	регистрация (BCrypt + парольная политика)

3.1. POST /auth/login — аутентификация

```
curl -X POST http://localhost:8080/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"Alice#Str0ng-2026"}'

{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJqdGkiOiI5MGJlZTFkZC00ODA5...",
  "tokenType": "Bearer",
  "expiresInSeconds": 900,
  "username": "alice",
  "roles": ["ROLE_USER"]
}
```

Неверная пара логин/пароль — 401 с обезличенным сообщением; 5 неудачных попыток подряд — 429 Too Many Requests с заголовком Retry-After.

3.2. GET /api/data — получение данных

```
curl "http://localhost:8080/api/data?page=0&size=3" -H "Authorization: Bearer $TOKEN"

{ "items": [ { "id": 1, "title": "OWASP Top 10 2021", "author": "admin", ... } ],
  "page": 0, "size": 3, "totalItems": 3, "totalPages": 1 }
```

Без заголовка Authorization эндпоинт отвечает 401:

```
{ "timestamp": "2026-09-10T16:50:39.531Z", "status": 401, "error": "unauthorized",
  "message": "Требуется действительный access-токен...", "path": "/api/data" }
```

3.3. POST /api/data — собственный метод

```
curl -X POST http://localhost:8080/api/data -H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"title": "XSS <script>alert(1)</script>",
  "content": "<img src=x onerror=alert(2)> plain text & symbols"}'

{ "id": 5, "title": "XSS", "content": "text and & symbol",
  "author": "alice", "createdAt": "2026-09-10T16:50:55.722Z" }
```

Скрипт и обработчик события вырезаны при сохранении, символ & возвращён в виде HTML-сущности. Автор берётся из проверенного токена, а не из тела запроса.

4. Реализованные меры защиты

4.1. Защита от SQL-инъекций (A03: Injection)

Конкатенация пользовательских данных с текстом SQL в проекте отсутствует. Используются два подхода.

а) ORM через Spring Data — запросы описаны JPQL с именованными параметрами:

```
@Query("""
SELECT p FROM Post p
WHERE LOWER(p.title) LIKE LOWER(CONCAT('%', :query, '%'))
""")
Page<Post> search(@Param("query") String query, Pageable pageable);
```

б) Явный PreparedStatement — SQL объявлен константой, значения передаются параметрами:

```
private static final String SEARCH_SQL = """
SELECT id, title, content, author_username, created_at
FROM posts
WHERE LOWER(title) LIKE LOWER(?) ESCAPE '\\'
ORDER BY created_at DESC LIMIT ?
""";

jdbcTemplate.query(SEARCH_SQL, ROW_MAPPER, pattern, pattern, limit);
```

Драйвер отправляет текст запроса и данные раздельно, поэтому кавычка внутри параметра остаётся обычным символом. Дополнительно экранируются служебные символы LIKE (% , _ \), иначе ввод % означал бы «выбрать всё».

Проверено:

Payload	Результат
' OR '1'='1 в /api/data/search	200, пустой массив — строка искалась как текст
x'; DROP TABLE posts; --	200, таблица цела, /api/data продолжает отдавать данные
admin'-- в поле логина	400 — отсечён валидацией до обращения к БД
' OR 1=1 -- в поле пароля	401 — обхода аутентификации нет

4.2. Защита от XSS (A03: Injection)

Реализованы два независимых слоя (класс HtmlSanitizer):

- Санитизация на входе — Jsoup с политикой Safelist.none() удаляет любые теги и атрибуты (script, img onerror, javascript:), в БД попадает только текст. Защита от хранимого (stored) XSS.
- Экранирование на выходе — OWASP Java Encoder применяется ко всем строковым полям в ответах API, поэтому спецсимволы уходят клиенту как HTML-сущности даже если данные попали в базу мимо API.

```
public String sanitize(String raw) { // вход
    String withoutMarkup = Jsoup.clean(raw, "", Safelist.none(), OUTPUT_SETTINGS);
    return Parser.unescapeEntities(withoutMarkup, false).trim();
}

public String encodeForHtml(String value) { // выход
    return Encode.forHtml(value);
}
```

Сопутствующие меры: ответы отдаются с Content-Type: application/json и заголовком X-Content-Type-Options: nosniff, CSP для API — default-src 'none'; sandbox.

Проверено:

Ввод	Что сохранилось и вернулось
XSS <script>alert('pwned')</script> Проверка	XSS Проверка
 Обычный текст & спецсимволы	Обычный текст & спецсимволы
клик	клик
<svg/onload=alert(1)>, <iframe src=...>	пустая строка — запрос отклоняется с 400

4.3. Аутентификация: JWT и BCrypt (A07)

Хеширование паролей. BCryptPasswordEncoder с версией \$2b и стоимостью 12. Открытый пароль не сохраняется, не логируется и не возвращается в API; toString() у DTO логина переопределён, чтобы пароль не попал в логи. В БД лежит строка вида \$2b\$12\$... — соль генерируется автоматически и хранится внутри хеша.

Выдача токена. После успешной проверки пары логин/пароль JwtService выпускает JWT, подписанный HS256:

```
{ "alg": "HS256" }
{
  "jti": "90bee1dd-8809-4b6c-89cf-66d3ee07b030",
  "sub": "alice",
  "iss": "cs-lab1-secure-api",
  "aud": ["cs-lab1-clients"],
  "iat": 1789059038, "nbf": 1789059038, "exp": 1789059938,
  "roles": ["ROLE_USER"]
}
```

Время жизни — 15 минут, секрет читается из переменной окружения APP_JWT_SECRET и обязан быть не короче 32 символов (256 бит для HS256), иначе приложение не стартует.

Middleware проверки токена. JwtAuthenticationFilter выполняется один раз на каждый запрос: достаёт токен только из заголовка Authorization: Bearer, проверяет его и кладёт аутентификацию в SecurityContext. Проверяются:

- HMAC-подпись — алгоритм задан явно, поэтому атака alg: none и подмена алгоритма не проходят;
- exp и nbf с допуском на расхождение часов 30 секунд;
- iss и aud — токен другого сервиса не подойдёт.

Все `/api/**` требуют аутентификации, неизвестные пути закрыты правилом `denyAll()` (`deny by default`). Ошибки отдаются в JSON: 401 — нет валидного токена, 403 — не хватает прав.

Защита от перебора. `LoginAttemptService` считает неудачные попытки по паре «логин + IP»: после 5 неудач вход блокируется на 15 минут, ответ — 429 с заголовком `Retry-After`. Успешный вход обнуляет счётчик.

Против перечисления пользователей. Ответ на «нет такого пользователя» и «неверный пароль» одинаков, а `DaoAuthenticationProvider` выполняет фиктивную проверку пароля для несуществующего логина, чтобы выровнять время ответа.

Проверено:

Сценарий	Ответ
Верные учётные данные	200 + токен (313 символов)
Неверный пароль / несуществующий пользователь	401, одинаковое сообщение
Запрос к <code>/api/data</code> без токена	401
Подделанная подпись, чужой ключ, просроченный токен	401
6 неудачных попыток подряд	401 пять раз, затем 429 + <code>Retry-After</code>

4.4. Разграничение доступа (A01: Broken Access Control)

- Deny by default: правило `anyRequest().denyAll()` — открыты только явно перечисленные маршруты.
- Проверка владельца ресурса: `DELETE /api/data/{id}` сверяет автора записи из БД с именем из проверенного токена; чужую запись удалить нельзя (403), что закрывает IDOR.
- RBAC: `GET /api/users` доступен только роли ADMIN — и на уровне маршрутов, и повторно через `@PreAuthorize` на методе сервиса (защита в глубину).
- Идентичность берётся из токена, а не из тела или параметров запроса.

4.5. Дополнительные меры

Мера	Реализация
Валидация ввода	Jakarta Bean Validation на всех DTO и query-параметрах: белые списки символов, границы длины, парольная политика (12+ символов, разные регистры, цифра, спецсимвол)
Security-заголовки	CSP, HSTS, X-Content-Type-Options, X-Frame-Options: DENY, Referrer-Policy, Permissions-Policy
CORS	белый список источников, без wildcard и без cookie-credentials
Обработка ошибок	единый JSON-формат, никаких стектрейсов и SQL наружу
Защита логов	LogSanitizer вырезает CR/LF из пользовательских значений — нельзя «дорисовать» строку в журнале
Секреты	JWT-секрет и пароли только из переменных окружения; в профиле prod значений по умолчанию нет
Минимум данных наружу	DTO не содержат хеш пароля; actuator открыт только на <code>/health</code>
Stateless	сессии не создаются, CSRF-защита не нужна (нет cookie-аутентификации)
Консоль H2	отключена

5. CI/CD pipeline с security-сканерами

Файл `.github/workflows/ci.yml`. Пайплайн запускается на каждый push и pull request в main, а также еженедельно по расписанию — чтобы находить уязвимости, опубликованные уже после мерджа.

Job	Инструмент	Тип	Блокирует сборку	Время
Сборка и автотесты	Maven + JUnit 5	—	да	47 с
SAST — SpotBugs + FindSecBugs	SpotBugs 4.9 + FindSecBugs 1.14	SAST	да, при любой находке	52 с
SCA — Trivy	Trivy 0.70 (сканирует собранный jar)	SCA	да, при HIGH/CRITICAL	42 с
SCA — OWASP Dependency-Check	dependency-check-maven 13	SCA	да, при $CVSS \geq 7.0$	2 мин 36 с
SAST — CodeQL	CodeQL, набор security-extended	SAST	нет	1 мин 20 с

6. Результаты работы сканеров

6.1. SAST: SpotBugs + FindSecBugs

Первый прогон с настройками `effort=Max`, `threshold=Low` дал 21 замечание. Все разобраны:

Детектор	Кол-во	Суть	Решение
CRLF_INJECTION_LOGS	6	в лог попадали данные, контролируемые клиентом — можно «дорисовать» поддельную строку в журнале	исправлено в коде: добавлен LogSanitizer
CT_CONSTRUCTOR_THROW	1	исключение в конструкторе JwtService оставляет объект частично инициализированным	исправлено в коде: класс объявлен final
EI_EXPOSE_REF / REP2	5	хранение и возврат изменяемых объектов	частично исправлено: SecurityProperties делает неизменяемую копию списка; для бинов Spring обосновано подавлено
SPRING_ENDPOINT	8	информационный детектор: «это точка входа, проверьте её»	подавлено с обоснованием — не дефект
SERVLET_HEADER	1	значение заголовка контролируется клиентом	подавлено: токен проверяется криптографически

Итог — 0 находок: BugInstance size is 0, BUILD SUCCESS. Все подавления с обоснованиями лежат в `config/spotbugs-exclude.xml`.

6.2. SCA: OWASP Dependency-Check

Первый полноценный прогон нашёл 46 уязвимостей в транзитивных зависимостях, из них 28 с $CVSS \geq 7.0$ (порог, на котором сборка падает).

1) Обновили до версий с исправлением — 17 находок закрыто:

Библиотека	Было	Стало	Закрыто
tomcat-embed-core	10.1.55	10.1.60	14 CVE, в т.ч. CVE-2026-65905 и CVE-2026-65637 (CVSS 9.8, обход аутентификации)

Библиотека	Было	Стало	Закрыто
log4j-api	2.24.3	2.25.5	CVE-2026-34479, CVE-2026-34477, CVE-2026-49844
jackson-databind	2.21.4	2.21.6	CVE-2026-54515
commons-lang3	3.17.0	3.20.0	CVE-2025-48924

2) Разобрали и обосновали — 14 находок в Spring Framework 6.2.19 и Spring Security 6.5.11.

Здесь SCA показал то, ради чего его и ставят: ветка, на которой стоит проект, перестала получать публичные патчи. Последние открытые релизы 6.2.x и 6.5.x вышли в июне 2026, а исправления этих CVE — только в Spring Framework 7.0.9 и Spring Security 7.x (август 2026), то есть в ветках Spring Boot 4.x. Поднять версию внутри 6.2.x невозможно: 6.2.19 и 6.5.11 — последние опубликованные.

Каждая находка проверена на применимость к приложению:

CVE	Уязвимая функция	Почему неприменимо здесь
CVE-2026-47892, -47891, -47893, -47889, -47888, -47885	WebFlux, RSocket, реактивный адаптер Jetty, парсер Aalto XML	этих модулей нет в classpath — проверено mvn dependency:list (93 runtime-зависимости)
CVE-2026-59313, -47890	SSE во функциональном веб-стеке	функциональных маршрутов и SSE нет, API отдаёт только JSON
CVE-2026-47884	SSRF/RCE через XsltView	представления не рендерятся вообще
CVE-2026-59283, -47886	вычисление пользовательского SpEL	SpEL только в константном @PreAuthorize
CVE-2026-59282	DoS через property paths в data binding	биндинга форм нет: тела запросов разбирает Jackson в record-DTO
CVE-2026-59270	встроенный LDAP-сервер UnboundIdContainer	spring-security-ldap и unboundid-ldapsdk отсутствуют
CVE-2026-47841	обход проверки в WebAuthn при распределённых сессиях	WebAuthn не настроен, приложение stateless

Записи лежат в config/dependency-check-suppressions.xml с атрибутом until="2026-12-31Z" — подавление истекает, и к разбору придётся вернуться: либо выйдет публичный патч, либо проект переедет на Spring Boot 4.x.

3) Оставили видимой — CVE-2026-47834 в spring-data-jpa 3.5.13 (CVSS 6.5). Она ниже порога, сборку не блокирует, публичного исправления для 3.5.x нет — поэтому не подавлена и остаётся в отчёте на виду.

Итоговый отчёт прогона № 12: просканировано 93 зависимости (60 уникальных), уязвимых — 3, найдено 8 уязвимостей, подавлено 14. Все 8 оставшихся имеют уровень MEDIUM или LOW (максимум 6.5), уязвимостей уровня HIGH и CRITICAL не осталось, поэтому job зелёный.

6.3. SCA: Trivy и SAST: CodeQL

Trivy 0.70.0 разбирает собранный fat-jar и сверяет все вложенные библиотеки с базой уязвимостей; порог — HIGH и CRITICAL, exit-code 1. Результат прогона № 12 — пустой список находок (results: [] в SARIF-отчёте).

CodeQL выполняет анализ набором запросов security-extended по коду, собранному командой mvnw compile. В прогоне № 12 успешно отработали все шаги: инициализация базы, сборка проекта и анализ. Замечаний не выдано.

7. Скриншоты отчётов SAST и SCA

Все изображения сняты с прогона № 12

(<https://github.com/PaulLocust/CS-lab1/actions/runs/35995043294>).

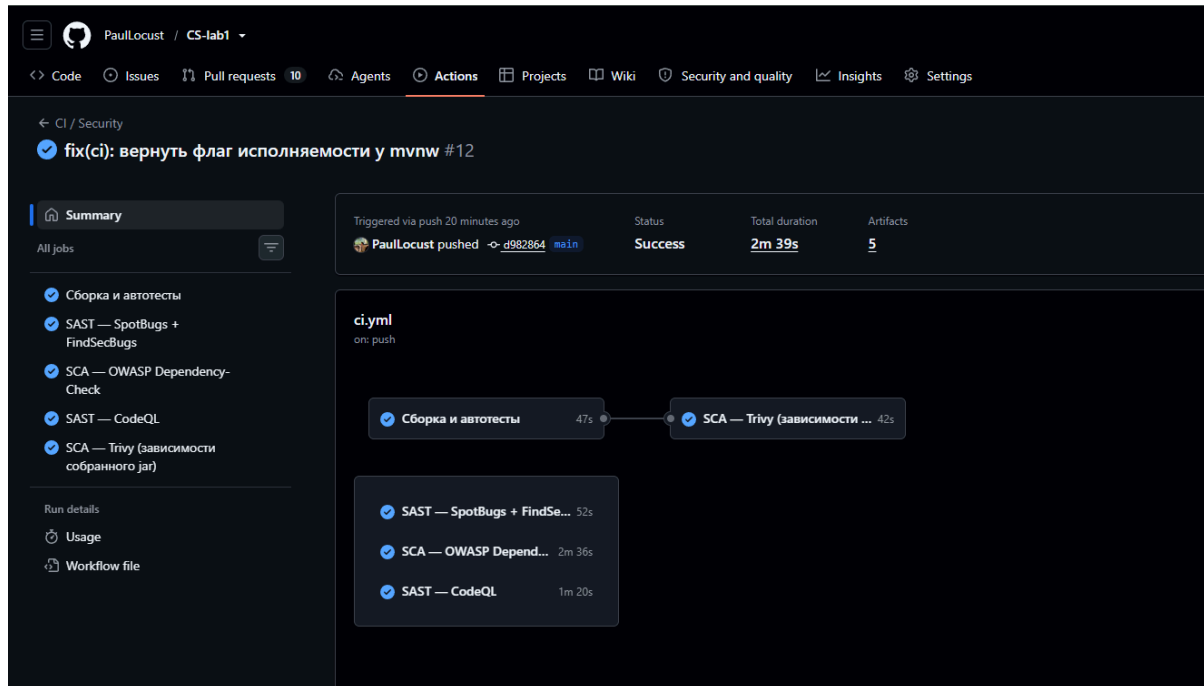


Рисунок 1 — Общий вид пайплайна: статус Success, все пять job зелёные, 2 мин 39 с

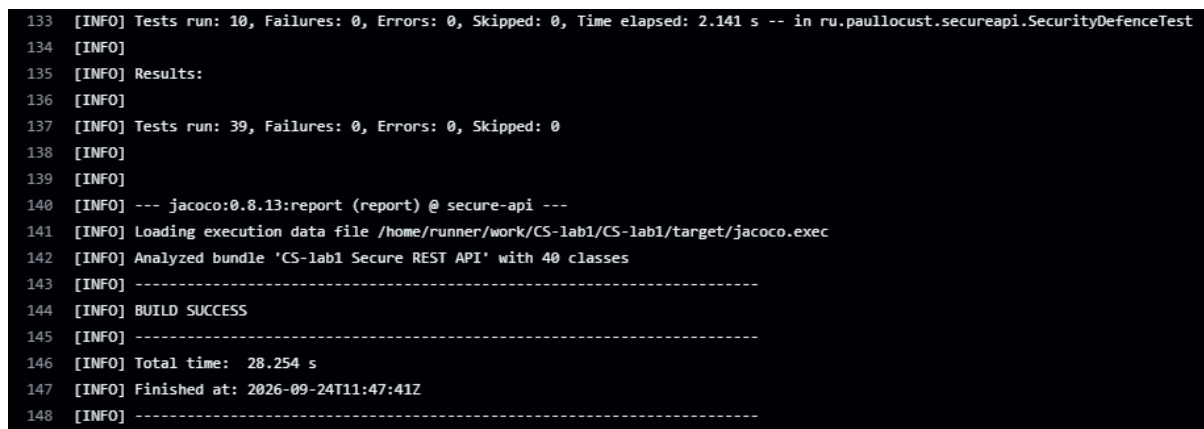


Рисунок 2 — Автотесты: Tests run: 39, Failures: 0, Errors: 0, Skipped: 0, BUILD SUCCESS



Рисунок 3 — SAST, SpotBugs + FindSecBugs: total_bugs="0" по всем 42 классам



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: CS-lab1 Secure REST API

ru.paullocust:secure-api:1.0.0

Scan Information ([show all](#)):

- dependency-check version: 13.0.0
- Report Generated On: Thu, 24 Sep 2026 11:49:24 GMT
- Dependencies Scanned: 93 (60 unique)
- Vulnerable Dependencies: 3
- Vulnerabilities Found: 8
- Vulnerabilities Suppressed: 14 ([show](#))
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
spring-core-6.2.19.jar	cpe:2.3:a:pivotal:software:spring:framework:6.2.19:***** cpe:2.3:a:springsource:spring:framework:6.2.19:***** cpe:2.3:a:vmware:spring:framework:6.2.19:*****	pkg:maven/org.springframework/spring-core@6.2.19	MEDIUM	5	Highest	41
spring-data-jpa-3.5.13.jar	cpe:2.3:a:pivotal:software:spring:data:jpa:3.5.13:***** cpe:2.3:a:vmware:spring:data:jpa:3.5.13:*****	pkg:maven/org.springframework.data/spring-data-jpa@3.5.13	MEDIUM	1	Highest	30
spring-security-core-6.5.11.jar	cpe:2.3:a:pivotal:software:spring:security:6.5.11:***** cpe:2.3:a:vmware:spring:security:6.5.11:*****	pkg:maven/org.springframework.security/spring-security-core@6.5.11	MEDIUM	2	Highest	38

Рисунок 4 — SCA, OWASP Dependency-Check: 93 зависимости, 8 находок уровня MEDIUM, 14 подавлено, HIGH и CRITICAL нет

```
1 {
2   "version": "2.1.0",
3   "$schema": "https://raw.githubusercontent.com/oasis-tcs/sarif-spec/main/sarif-2.1/schema/sarif-schema-2.1.0.json",
4   "runs": [
5     {
6       "tool": {
7         "driver": {
8           "fullName": "Trivy Vulnerability Scanner",
9           "informationUri": "https://github.com/aquasecurity/trivy",
10          "name": "Trivy",
11          "rules": [],
12          "version": "0.70.0"
13        }
14      },
15      "results": [],
16      "columnKind": "utf16CodeUnits",
17      "originalUriBaseIds": {
18        "ROOTPATH": {
19          "uri": "file:///home/runner/work/CS-lab1/CS-lab1/target/"
20        }
21      }
22    }
23  ]
24 }
```

Рисунок 5 — SCA, Trivy: пустой список результатов — уязвимостей HIGH и CRITICAL не найдено

SAST — CodeQL

succeeded 34 minutes ago in 1m 20s

Q Search logs

🔄 ⚙️

> Set up job

> Получить исходный код

> Установить JDK 21

> Инициализировать CodeQL

> Собрать проект для анализа

> Анализ CodeQL

> Post Анализ CodeQL

> Post Инициализировать CodeQL

> Post Установить JDK 21

> Post Получить исходный код

> Complete job

3s

0s

2s

14s

17s

38s

1s

0s

1s

0s

0s

8. Тестирование

Написано 39 автотестов (JUnit 5 + MockMvc). 23 из них целиком посвящены механизмам защиты, остальные проверяют функциональность API и разграничение доступа.

Класс тестов	Тестов	Что проверяет
AuthApiTest	7	выдача JWT, 401 при неверных данных, одинаковый ответ для несуществующего пользователя, валидация, блокировка после серии неудач, регистрация и парольная политика
DataApiTest	9	401 без токена, 200 с токеном, создание записи, 403 при удалении чужой записи, 204 для автора, 403 на админский эндпоинт для обычного пользователя
SecurityDefenceTest	10	XSS-payload, три сценария SQL-инъекций, подделанный, чужой и просроченный токен, security-заголовки, deny by default
HtmlSanitizerTest	8	вырезание script, onerror, svg/onload, iframe, экранирование спецсимволов
JwtServiceTest	5	корректность claim-ов, отсутствие пароля в payload, отказ при подделке, чужом издателе и коротком секрете

Ручная проверка выполнялась через curl и Postman: в репозитории лежат скрипты scripts/smoke-test.sh и scripts/smoke-test.ps1 (13 сценариев), коллекция postman/CS-lab1.postman_collection.json и файл docs/api-examples.md с реальными ответами сервиса.